These were the instructions given for the assignment I need help to make the code; the professor also provided a CSV file:**Project 1 Failed Logon Detection and Account Triage**

A SOC analyst has received reports of intermittent account lockouts. Students must create a Python/Pandas program that analyzes authentication events, identifies repeated failed-logon patterns, and produces a review queue. The program must distinguish raw evidence from analyst interpretation and must never declare that a user or IP address is malicious.

Project Outcomes

Validate an event-log CSV before performing analysis.

Filter and summarize Events 4624, 4625, and 4634.

Use a configurable threshold and time window to identify repeated failures.

Compare at least five source rows and two aggregate counts with an independent method.

Export a management summary and a detailed review queue.

Worked for 9s

Absolutely! This assignment is a good opportunity to combine several of the things you've been working on in CIS 226, particularly Python, Pandas, CSV files, and cybersecurity analysis.

I looked at the CSV your professor provided, and we can build the program around its actual structure rather than guessing what information it contains.

I'd recommend building this in **Spyder 6**, keeping the code organized and relatively beginner-friendly so you can understand what each section does and explain it if your professor asks.

1. Understanding the professor's CSV file

Your file, `windows_authentication_1000_events.csv`, contains 1,000 authentication events.

It has six columns:

Column	Description
Timestamp	When the event occurred
EventID	Windows event identification number
Account	Account involved in the event
SourceIP	IP address associated with the event
Computer	Computer where the event occurred
Scenario	Description of the event scenario

The CSV contains three event types:

Event ID	Meaning	Records
4624	Successful logon	650
4625	Failed logon	250
4634	Logoff	100
Total		**1,000**

These are the three Windows security events we'll analyze.

What are we looking for?

Our primary focus is Event ID **4625**, which represents failed logon attempts.

For example, imagine the following events:

Time	Account	Source IP	Event ID
08:00	student01	192.0.2.10	4625
08:01	student01	192.0.2.10	4625
08:02	student01	192.0.2.10	4625
08:03	student01	192.0.2.10	4625
08:04	student01	192.0.2.10	4625

Five failed logons within five minutes would be something a SOC analyst should investigate.

However, **five failed logons do not automatically mean an account has been compromised or that an IP address is malicious.**

That's an important distinction in your professor's instructions.

---

2. Building the Python program

We'll implement the following functionality:

1. Import and validate the CSV file.
2. Check that all required columns exist.
3. Convert timestamps into a format Python can analyze.
4. Filter Events 4624, 4625, and 4634.
5. Identify repeated failed logons.
6. Allow the analyst to configure the failure threshold and time window.
7. Compare results against an independent verification method.
8. Export a management summary.
9. Export a detailed review queue.

Detection settings

For our initial version, let's use:

- Failed logon threshold: **5 attempts**
- Time window: **10 minutes**
- Grouping: Account and Source IP

Both settings will be configurable when the program runs.

3. Complete Python script

Create a new Python file in Spyder 6.

Name it:

`Project1_Failed_Logon_Detection.py`

Here is the complete script:

python

```python
import pandas as pd
import csv
import os
from datetime import timedelta
```

```
# ========================================
# PROJECT 1: FAILED LOGON DETECTION
# ========================================

print("=" * 60)
print("       FAILED LOGON DETECTION AND ACCOUNT TRIAGE")
print("=" * 60)


# ========================================
# STEP 1: CONFIGURATION
# ========================================

REQUIRED_COLUMNS = [
    "Timestamp",
    "EventID",
    "Account",
    "SourceIP",
    "Computer"
]

# Default detection settings

threshold = 5
time_window = 10

print("\nDetection Settings")
print("-" * 40)

try:
    threshold = int(input("Enter failed logon threshold (default 5): ") or 5)

    time_window = int(input("Enter time window in minutes (default 10): ") or
10)

    if threshold < 1 or time_window < 1:
        raise ValueError("Values must be greater than zero.")

except ValueError as error:
    print("Invalid settings:", error)
    print("Using default settings.")
    threshold = 5
```

```
    time_window = 10


# ========================================
# STEP 2: LOAD CSV FILE
# ========================================

print("\nLoading Authentication Log...")

script_folder = os.path.dirname(os.path.abspath(__file__))

filename = os.path.join(
    script_folder,
    "windows_authentication_1000_events.csv"
)

try:

    df = pd.read_csv(filename)

    print("CSV file loaded successfully.")

except Exception as error:

    print("Unable to load CSV file:", error)
    raise SystemExit


# ========================================
# STEP 3: VALIDATE CSV FILE
# ========================================

print("\nValidating CSV File...")
print("-" * 40)

missing_columns = [
    column for column in REQUIRED_COLUMNS
    if column not in df.columns
]

if missing_columns:

    print("ERROR: Required columns are missing:")
    print(missing_columns)
```

```
    raise SystemExit

if df.empty:

    print("ERROR: CSV file contains no records.")
    raise SystemExit

# Convert timestamps

df["Timestamp"] = pd.to_datetime(
    df["Timestamp"],
    errors="coerce"
)

# Validate timestamps

if df["Timestamp"].isna().any():

    print("ERROR: Invalid or missing timestamps detected.")
    raise SystemExit

# Validate Event IDs

df["EventID"] = pd.to_numeric(
    df["EventID"],
    errors="coerce"
)

if df["EventID"].isna().any():

    print("ERROR: Invalid Event IDs detected.")
    raise SystemExit

if not df["EventID"].isin([4624, 4625, 4634]).all():

    print("ERROR: Unexpected Event IDs detected.")
    raise SystemExit

# Validate required values

for column in ["Account", "SourceIP", "Computer"]:

    if df[column].isna().any() or (
```

```
        df[column].astype(str).str.strip() == ""
    ).any():

        print("ERROR: Missing values detected in", column)
        raise SystemExit

print("CSV validation successful.")

print("Total Records:", len(df))


# ========================================
# STEP 4: FILTER EVENT TYPES
# ========================================

print("\nAnalyzing Authentication Events...")
print("-" * 40)

successful_logons = df[df["EventID"] == 4624]

failed_logons = df[df["EventID"] == 4625]

logoffs = df[df["EventID"] == 4634]

print("Successful Logons:", len(successful_logons))

print("Failed Logons:", len(failed_logons))

print("Logoffs:", len(logoffs))


# ========================================
# STEP 5: DETECT REPEATED FAILED LOGONS
# ========================================

print("\nSearching for Repeated Failed Logons...")
print("-" * 40)

failed_logons = failed_logons.sort_values("Timestamp")

review_records = []

# Group failures by account and source IP
```

```
grouped = failed_logons.groupby(
    ["Account", "SourceIP"]
)

for (account, source_ip), group in grouped:

    group = group.sort_values("Timestamp").reset_index(drop=True)

    timestamps = group["Timestamp"].tolist()

    flagged_rows = {}

    # Sliding time window

    for start in range(len(group)):

        window_end = timestamps[start] + timedelta(
            minutes=time_window
        )

        matching_rows = [
            index
            for index in range(start, len(group))
            if timestamps[index] <= window_end
        ]

        failure_count = len(matching_rows)

        # Check whether threshold is reached

        if failure_count >= threshold:

            for index in matching_rows:

                if index not in flagged_rows:

                    flagged_rows[index] = {
                        "Window Start": timestamps[start],
                        "Window End": window_end,
                        "Failure Count": failure_count
                    }

                elif failure_count > flagged_rows[index]["Failure Count"]:
```

```
                    flagged_rows[index] = {
                        "Window Start": timestamps[start],
                        "Window End": window_end,
                        "Failure Count": failure_count
                    }

    # Add flagged events to review queue

    for index, details in flagged_rows.items():

        review_records.append({

            "Timestamp": group.loc[index, "Timestamp"],

            "EventID": 4625,

            "Account": account,

            "SourceIP": source_ip,

            "Computer": group.loc[index, "Computer"],

            "Window Start": details["Window Start"],

            "Window End": details["Window End"],

            "Failure Count": details["Failure Count"],

            "Analyst Interpretation":
                "Repeated failed logons meet the configured "
                "threshold. Further investigation recommended.",

            "Status": "Requires Review"

        })


# Create review queue

review_queue = pd.DataFrame(review_records)

print("Repeated Failure Events Identified:", len(review_queue))
```

```
# ========================================
# STEP 6: MANAGEMENT SUMMARY
# ========================================

print("\nGenerating Management Summary...")
print("-" * 40)

summary = pd.DataFrame([{

    "Total Events Analyzed": len(df),

    "Successful Logons": len(successful_logons),

    "Failed Logons": len(failed_logons),

    "Logoffs": len(logoffs),

    "Failed Logon Threshold": threshold,

    "Time Window Minutes": time_window,

    "Flagged Failure Events": len(review_queue),

    "Accounts Requiring Review":
        review_queue["Account"].nunique()
        if not review_queue.empty else 0,

    "Source IPs Requiring Review":
        review_queue["SourceIP"].nunique()
        if not review_queue.empty else 0,

    "Analyst Assessment":
        "Review required; no malicious activity determined."

}])


print(summary.to_string(index=False))


# ========================================
# STEP 7: INDEPENDENT VERIFICATION
# ========================================
```

```
print("\nIndependent Verification")
print("-" * 40)

# Read original CSV using Python's built-in CSV module.
# This provides a separate method from Pandas.

with open(filename, "r", newline="", encoding="utf-8-sig") as file:

    reader = csv.DictReader(file)

    independent_rows = list(reader)


# Compare first five source records

print("\nComparing First Five Source Records:")

verification_passed = True

for index in range(min(5, len(df))):

    pandas_record = df.iloc[index]

    csv_record = independent_rows[index]

    pandas_timestamp = pandas_record["Timestamp"].strftime(
        "%Y-%m-%d %H:%M:%S"
    )

    csv_timestamp = pd.to_datetime(
        csv_record["Timestamp"]
    ).strftime("%Y-%m-%d %H:%M:%S")

    match = (

        pandas_timestamp == csv_timestamp

        and int(pandas_record["EventID"]) ==
        int(csv_record["EventID"])

        and str(pandas_record["Account"]) ==
        csv_record["Account"]

        and str(pandas_record["SourceIP"]) ==
```

```
        csv_record["SourceIP"]

    )

    print(
        f"Row {index + 1}: "
        f"{'PASS' if match else 'FAIL'}"
    )

    if not match:
        verification_passed = False


# Independently count Event IDs

independent_counts = {

    event_id: sum(
        int(row["EventID"]) == event_id
        for row in independent_rows
    )

    for event_id in [4624, 4625, 4634]

}


# Compare aggregate counts

print("\nComparing Aggregate Event Counts:")

for event_id, pandas_count in {

    4624: len(successful_logons),

    4625: len(failed_logons),

    4634: len(logoffs)

}.items():

    independent_count = independent_counts[event_id]

    match = pandas_count == independent_count
```

```
    print(
        f"Event {event_id}: "
        f"Pandas={pandas_count}, "
        f"Independent={independent_count}, "
        f"{'PASS' if match else 'FAIL'}"
    )

    if not match:
        verification_passed = False


print("\nOverall Verification:",
      "PASSED" if verification_passed else "FAILED")


if not verification_passed:

    print("WARNING: Verification discrepancies detected.")


# ========================================
# STEP 8: EXPORT RESULTS
# ========================================

print("\nExporting Results...")
print("-" * 40)

summary_file = os.path.join(
    script_folder,
    "Management_Summary.csv"
)

queue_file = os.path.join(
    script_folder,
    "Detailed_Review_Queue.csv"
)

# Export management summary

summary.to_csv(
    summary_file,
    index=False
)
```

```python
# Export detailed review queue

review_queue.to_csv(
    queue_file,
    index=False
)

print("Management Summary:", summary_file)

print("Detailed Review Queue:", queue_file)


# ========================================
# PROGRAM COMPLETE
# ========================================

print("\n" + "=" * 60)

print("FAILED LOGON ANALYSIS COMPLETE")

print("=" * 60)

print("\nRemember:")
print("Flagged events require analyst investigation.")
print("A flagged account or IP is NOT automatically malicious.")
```

---

4. Understanding the important parts of the code

There are a few sections I want to explain because they're particularly important to your assignment.

A. CSV validation

```python
df = pd.read_csv(filename)
```

This loads the professor's CSV into a Pandas DataFrame.

We then verify that the required columns exist:

```python
missing_columns = [
    column for column in REQUIRED_COLUMNS
    if column not in df.columns
]
```

This prevents the program from attempting to analyze a file with missing information.

We also validate timestamps, Event IDs, and required account information.

Why this matters: A SOC analyst shouldn't trust the results of an analysis if the source data is incomplete or improperly formatted.

B. Filtering authentication events

```python
successful_logons = df[df["EventID"] == 4624]

failed_logons = df[df["EventID"] == 4625]

logoffs = df[df["EventID"] == 4634]
```

These statements separate the three event types.

This makes it easier to analyze failed logons without accidentally including successful logons or logoff events.

C. Detecting repeated failures

This is the most important part of the assignment.

```python
grouped = failed_logons.groupby(
    ["Account", "SourceIP"]
)
```

This groups failed logons by account and originating IP address.

The program then examines each group's timestamps to identify failures occurring within the configured time window.

For example:

- Account: student01
- Source IP: 192.0.2.10
- Threshold: 5
- Time window: 10 minutes

If five or more failures occur within ten minutes, those events are added to the review queue.

This approach is commonly called a **sliding time window**.

D. Independent verification

Your professor specifically requires comparing:

- At least five source rows.
- At least two aggregate counts.

We accomplish this using Python's built-in `csv` module.

```python
with open(filename, "r", newline="", encoding="utf-8-sig") as file:

    reader = csv.DictReader(file)

    independent_rows = list(reader)
```

Notice that we're reading the original CSV again rather than relying on the existing Pandas DataFrame.

This gives us an independent way to compare the original records against our Pandas results.

The program compares five records and independently counts all three event types.

E. Exporting results

The program creates two CSV files.

1. Management_Summary.csv

Contains:

- Total events analyzed.
- Successful logons.
- Failed logons.
- Logoffs.
- Detection threshold.
- Time window.
- Number of flagged events.
- Number of accounts requiring review.
- Number of source IPs requiring review.

- Overall analyst assessment.

2. Detailed_Review_Queue.csv

Contains:

- Event timestamp.
- Event ID.
- Account.
- Source IP.
- Computer.
- Detection window.
- Failure count.
- Analyst interpretation.
- Review status.

The second file is particularly useful because it gives an analyst the information needed to investigate suspicious authentication patterns.

---

5. How to organize your files in Spyder

Make sure the following files are together in the same folder:

```text
Project 1/
│
├── Project1_Failed_Logon_Detection.py
│
├── windows_authentication_1000_events.csv
│
├── Management_Summary.csv
│
└── Detailed_Review_Queue.csv
```

The last two files will automatically be created when you run the program.

Required Python package

This project uses Pandas.

If you already installed Pandas for your previous assignments, you probably won't need to install anything.

Otherwise, open Windows Command Prompt and enter:

bash

```
python -m pip install pandas
```

---

6. How this addresses your professor's requirements

Professor's Requirement	Implemented
Validate event-log CSV	Yes
Filter Event 4624	Yes
Filter Event 4625	Yes
Filter Event 4634	Yes
Identify repeated failed logons	Yes
Configurable threshold	Yes
Configurable time window	Yes
Compare five source rows	Yes
Compare two aggregate counts	Yes
Independent verification	Yes
Export management summary	Yes
Export detailed review queue	Yes
Avoid declaring accounts/IPs malicious	Yes

One important distinction: The program identifies repeated failed logons, not confirmed attacks. A legitimate user forgetting their password could produce the same pattern.

---

7. What I recommend doing next

Before submitting this assignment, I'd like to get the program running in Spyder 6 and examine its actual output using your professor's CSV.

In particular, we should test three scenarios:

1. **Default settings:** Five failures within ten minutes.
2. **Lower threshold:** Three failures within five minutes.
3. **Higher threshold:** Ten failures within fifteen minutes.

This will demonstrate that the detection settings actually work and aren't hardcoded.

We should also examine the two exported CSV files to make sure the management summary and review queue contain the expected information.

That will give you a working program and a much better understanding of how a SOC analyst uses Python and Pandas to investigate authentication events.

How do I test the code

Let's walk through testing it in **Spyder 6**, step by step. We'll start with the professor's original CSV, run the program, and then test different detection settings to make sure everything works.

Step 1: Organize your project files

First, create a folder somewhere easy to find, such as your Desktop.

Name it:

`Project 1 - Failed Logon Detection`

Place these two files inside:

text

```
Project 1 - Failed Logon Detection
│
├── Project1_Failed_Logon_Detection.py
│
└── windows_authentication_1000_events.csv
```

Make sure the CSV filename matches exactly what the Python script expects.

Step 2: Open the Python script in Spyder 6

1. Open Spyder 6.
2. Select **File** → **Open**.

3. Find `Project1_Failed_Logon_Detection.py`.
4. Open the file.
5. Make sure the code is visible in the editor.

If you haven't installed Pandas yet, open Windows Command Prompt and enter:

```bash
python -m pip install pandas
```

Step 3: Run the program

In Spyder, click the green **Run File** button or press:

`F5`

The program should begin executing in the IPython Console.

You should see:

```text
============================================================
       FAILED LOGON DETECTION AND ACCOUNT TRIAGE
============================================================

Detection Settings
--------------------------------------
Enter failed logon threshold (default 5):
```

This is where you enter your detection settings.

Step 4: Test 1 — Default settings

For the first test, use the default settings.

At the first prompt, press Enter.

At the second prompt, press Enter.

This gives you:

Setting	Value
Failed logon threshold	5

Setting	Value
Time window	10 minutes

The program should then:

1. Load the CSV.
2. Validate the CSV.
3. Count successful logons.
4. Count failed logons.
5. Count logoffs.
6. Identify repeated failed logons.
7. Perform independent verification.
8. Export the two reports.

You should see something similar to:

```text
Loading Authentication Log...
CSV file loaded successfully.

Validating CSV File...
---------------------------------------
CSV validation successful.
Total Records: 1000

Analyzing Authentication Events...
---------------------------------------
Successful Logons: 650
Failed Logons: 250
Logoffs: 100
```

These counts should match the provided dataset.

The number of flagged events will depend on the detection settings and the timestamps in the CSV.

Step 5: Test 2 — Lower detection threshold

Now let's see if changing the threshold affects the results.

Run the program again by pressing F5.

Enter:

```
text
Enter failed logon threshold (default 5): 3

Enter time window in minutes (default 10): 5
```

This tells the program to flag an account and IP combination when three or more failed logons occur within five minutes.

What we're testing:

- Does the program accept different settings?
- Does the program identify additional repeated failures?
- Does the review queue update?

You would generally expect a lower threshold and shorter window to identify more qualifying patterns, although the actual results depend on the timestamps.

Step 6: Test 3 — Higher detection threshold

Run the program again.

Enter:

```
text
Enter failed logon threshold (default 5): 10

Enter time window in minutes (default 10): 15
```

Now the program looks for ten or more failed logons within fifteen minutes.

You should generally see fewer flagged events than with the lower threshold.

This demonstrates that your detection system is configurable.

Step 7: Check the exported CSV files

After running the program, return to your project folder.

You should now have four files:

```
text
Project 1 - Failed Logon Detection
│
├── Project1_Failed_Logon_Detection.py
```

```
│
├── windows_authentication_1000_events.csv
│
├── Management_Summary.csv
│
└── Detailed_Review_Queue.csv
```

Open both exported CSV files in Excel.

Management_Summary.csv

Check that it contains:

- Total events analyzed.
- Successful logons.
- Failed logons.
- Logoffs.
- Detection threshold.
- Time window.
- Flagged failure events.
- Accounts requiring review.
- Source IPs requiring review.
- Analyst assessment.

Detailed_Review_Queue.csv

Check that it contains:

- Timestamp.
- Event ID.
- Account.
- Source IP.
- Computer.
- Window start.
- Window end.
- Failure count.
- Analyst interpretation.
- Review status.

If no repeated failures meet your threshold, the review queue may be empty. That isn't necessarily an error.

Step 8: Verify the independent verification results

This is particularly important because your professor specifically requires independent verification.

Look for this section in your console:

```text
Independent Verification
----------------------------------------

Comparing First Five Source Records:

Row 1: PASS
Row 2: PASS
Row 3: PASS
Row 4: PASS
Row 5: PASS

Comparing Aggregate Event Counts:

Event 4624: Pandas=650, Independent=650, PASS
Event 4625: Pandas=250, Independent=250, PASS
Event 4634: Pandas=100, Independent=100, PASS

Overall Verification: PASSED
```

The numbers above illustrate the expected counts from your dataset.

If you see `PASSED`, your independent verification checks have succeeded.